

Minisweeper System Architecture

This document describes the design layout of team 22's Minesweeper program

1. Overview

The game presents 3 main screens for the player: title, settings and game. `main.py` contains the simple state machine that controls the pygame application's screen. `board.py` via `Board` class manages all the game play action: mine placement, revealing, flagging, win/loss, which has no dependency on pygame. A reusable `Button` widget (`button.py`) handles all clickable UI elements across every screen.

```
minesweeper/
|— main.py      # App shell: state machine, rendering, input routing
|— board.py     # Game engine: grid state + rules (no UI dependency)
|— button.py    # Reusable clickable UI widget
```

Our team implementation merged `Board Manager` and `Game Logic` into one `Board` class, since grid state and the rules that mutate it are tightly coupled and simplest to keep together. `User Interface` and `Input Handler` are mapped directly onto `main.py`.

2. Components

2.1 Board (`board.py`) — Board Manager + Game Logic

Manage all grid states and rules of the game.

Method	Purpose
<code>__init__(size, mine_count)</code>	Allocates the grid state for a $\text{size} \times \text{size}$ board. Mines are not placed yet.
<code>reveal(r, c)</code>	Reveals a cell. On the <i>first</i> call, places mines. If the cell is a mine, ends the game as a loss. If it has zero adjacent mines, flood-fills outward via an explicit stack. Checks for a win after every reveal.
<code>chord(r, c)</code>	On an already-revealed numbered cell, if the number of flags around it matches its count, it reveals all remaining unflagged neighbors.
<code>toggle_flag(r, c)</code>	Flags/unflags a covered cell. Refuses to add a flag once the flag count equals <code>mine_count</code> to prevent over-flagging.
<code>flags_placed()</code>	Returns the current number of flags on the board.

<code>_neighbors(r, c)</code>	Generator yielding the up-to-8 valid neighbor coordinates of a cell.
<code>_place_mines(safe_r, safe_c)</code>	Randomly places <code>mine_count</code> mines, excluding the given cell and its neighbors, then computes adjacency counts for every cell.

Design notes:

- Mine placement is deferred until the first `reveal()` call, and explicitly excludes the clicked cell *and its neighbors* to guarantee the player's first click always opens a safe area, per the spec's requirement.
- `reveal()`'s flood fill is iterative stack-based, not recursive to avoid Python's recursion limit on boards with large empty regions.

2.2 Button (`button.py`) — UI widget

Simple class that draws the rectangles that make up the UI elements.

Method	Purpose
<code>__init__(rect, text, font)</code>	Defines the button's position/size, label, and font.
<code>draw(surface)</code>	Renders the button, swapping to a lighter fill color when the mouse is hovering over it.
<code>is_clicked(pos)</code>	Returns True if a given point falls inside the button.

2.3 Game (`main.py`) — App shell: Input Handler + User Interface

The application's entry point and controller that handles both the UI display and the actions of the user.

As Input Handler, `Game.handle_event()` receives every pygame event: mouse clicks, key presses, quit, and routes it based on the current screen:

- `handle_title_click()` — Play / Quit
- `handle_settings_click()` — adjust mine count, Start, Back
- `handle_game_click()` — converts a pixel click into a (row, col) grid coordinate, then calls `board.reveal()`, `board.chord()`, or `board.toggle_flag()` depending on which mouse button was used and whether the cell is already revealed

As User Interface, `Game.draw()` dispatches to one of three renderers based on `self.state`:

- `draw_title()` / `draw_settings()` — static screens with Button widgets
- `draw_game()` → `draw_hud()` (mine counter, timer, Reset/Menu buttons) and `draw_board()` (renders all 100 cells by reading directly from `self.board`'s state each frame)
- `draw_game_over_overlay()` — win/loss message shown on top of the board

Application state machine — three states, stored in `self.state`. `start_new_game()` is the bridge between the UI and the engine: it constructs a fresh `Board(GRID_SIZE, self.mine_count)` and resets the timer.

State	Meaning	Transitions out
STATE_TITLE	Landing screen	Play → Settings, Quit → exit
STATE_SETTINGS	Choose mine count (10–20)	Start → Game (calls <code>start_new_game()</code>), Back → Title
STATE_GAME	Active gameplay	Menu button or Esc → Title; R/M after game over → restart/menu

3. Data Flow

3.1 Example trace: a left-click on a covered cell

1. Player left-clicks inside the grid.
2. `handle_game_click()` converts the pixel position into (row, col) and, since the cell isn't yet revealed, calls `board.reveal(row, col)`.
3. If this is the first reveal of the game, `Board._place_mines()` runs first, seeding mines everywhere except the clicked cell and its neighbors.
4. If the cell is a mine: `revealed[r][c] = True`, `game_over = True`, `won = False`.
5. Otherwise: an iterative flood-fill reveals the cell (and, if it has zero adjacent mines, cascades outward to neighboring cells the same way).
6. Board checks whether `revealed_count` has reached every non-mine cell; if so, `game_over = True`, `won = True`.
7. Control returns to the main loop. On the next frame, `draw_board()` reads `board.revealed`, `board.flagged`, `board.counts`, and `board.mines` directly to render the updated grid. There's no separate "event" or "diff" system; the UI simply re-reads state every frame.

3.2 Right-click (flagging) and chording

- **Right-click** → `board.toggle_flag(row, col)` → flips `flagged[r][c]`, refusing to add a flag once `flags_placed() == mine_count`.
- **Left-click on an already-revealed numbered cell** → `board.chord(row, col)` → if the surrounding flag count matches the cell's number, reveal all remaining unflagged neighbors by calling `reveal()` on each.

4. Key Data Structures

All grid state lives on the `Board` instance; `Game` holds only app-level state.

Name	Location	Type	Purpose
mines	Board	set of (row, col) tuples	Locations of all mines
counts	Board	10×10 list[list[int]]	Number of adjacent mines per cell (0–8)
revealed	Board	10×10 list[list[bool]]	Whether each cell has been uncovered
flagged	Board	10×10 list[list[bool]]	Whether each cell has a flag
mines_placed	Board	bool	Whether mine placement has happened yet (deferred to first click)
game_over	Board	bool	Whether the game has ended (win or loss)
won	Board	bool	Outcome once game_over is True
revealed_count	Board	int	Running count of uncovered non-mine cells, used for win detection
state	Game	str ("title" / "settings" / "game")	Which screen is currently active
mine_count	Game	int (10–20)	Player-configured mine count, set in Settings
board	Game	Board instance or None	The active game session; None until start_new_game() runs

Our implementation differs from the spec's suggested design of a single 2D array with coded values (0=covered, 1=flagged, 2=uncovered, 3=mine). The team decided to instead split state across four parallel 10×10 structures plus a set for mine locations. This trades a bit of memory for clearer, more directly-testable logic, for example if (r, c) in self.mines reads more plainly than comparing against a magic number.
